# Automated QoR Improvement in OpenROAD with Coding Agents

Amur Ghose
aghose@ucsd.edu
UC San Diego
La Jolla, CA, USA

Junyeong Jang
juj015@ucsd.edu
UC San Diego
La Jolla, CA, USA

Andrew B. Kahng
abk@ucsd.edu
UC San Diego
La Jolla, CA, USA

Jakang Lee
jal216@ucsd.edu
UC San Diego
La Jolla, CA, USA

## Abstract

Electronic design automation (EDA) tool development is limited by the scarcity of engineers who can modify large production codebases. Large language models (LLMs) perform well on coding tasks, but it remains unclear whether they can improve the algorithms inside production EDA tools. We present **AuDoPEDA**, a repository-grounded coding system that reads OpenROAD, generates research directions, expands them into implementation steps, and submits executable diffs (line-by-line code changes). Our contributions include (i) an autonomous system that modifies production EDA source code under closed-loop quality-of-results (QoR) feedback; (ii) a task suite and evaluation protocol on OpenROAD for QoR-oriented improvements; (iii) end-to-end results on OpenROAD where one diff per optimization target is reused across multiple designs, with the suite spanning three process design kits (PDKs); and (iv) a formal verification gate that attaches machine-checked Lean proofs to selected diffs. Experiments in OpenROAD achieve routed wirelength reductions of up to 5.9%, effective clock period (ECP) reductions of up to 10.0%, and power reductions of up to 19.4%.

## Keywords

OpenROAD, coding agents, physical design, QoR optimization, formal verification

**ACM Reference Format:**
Amur Ghose, Junyeong Jang, Andrew B. Kahng, and Jakang Lee. 2026. Automated QoR Improvement in OpenROAD with Coding Agents. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '26), November 08–12, 2026, San Jose, CA, USA.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3831252.3834240

## 1 Introduction

VLSI physical design (PD) remains heavily constrained by the availability of senior engineers who can reason across large, multi-language electronic design automation (EDA) codebases and long, iterative flows. Recent code-focused large language models (LLMs) show strong performance on local programming tasks, but their effectiveness degrades when the required context spans many files, undocumented invariants, and mixed-language toolchains. The open-source OpenROAD project [1, 2] exemplifies this setting: it comprises millions of lines of C++, Tcl, Python and Verilog, evolves on a weekly cadence, and offers only sparse cross-module documentation with many implicit interfaces. These characteristics increase development complexity and pose an even greater challenge for autonomous coding agents.

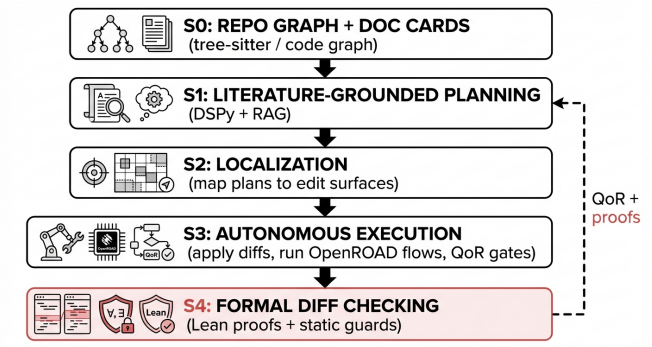


**Figure 1: The AuDoPEDA pipeline (with an optional formal verification gate).**

In this work, we study whether an LLM-driven system can make code changes across an entire production EDA repository and automatically verify that those changes improve physical-design quality-of-results (QoR). ORFS-agent [17] tunes predefined flow knobs. Section 5 compares that approach with source editing under a matched budget. AuDoPEDA generates structured documentation and a code graph for OpenROAD, retrieves from both repository docs and EDA literature, and applies localized diffs (line-by-line code changes) through an agent that compiles, runs flows, and accepts only non-regressing (i.e., non-worsening) changes under fixed evaluation gates. We decompose the task into four stages plus an optional formal verification gate:

**Stage 0: Documentation bootstrap and code graph.** We parse the OpenROAD repository with *tree-sitter* [3] to build a pruned code graph and generate per-module documentation cards for downstream retrieval [4, 5].

**Stage 1: Literature-grounded, DSPy-programmed planning.** We use *DSPy* [6] to structure the planner as a multi-step LLM program rather than a single prompt. The planner retrieves from (a) the Docmaker cards and code graph, and (b) a domain documentation set covering placement, routing, and timing closure [4]. It outputs a structured research plan (objective, hypotheses, target files, expected metrics and validation criteria) so that each plan is directly executable and auditable.

**Stage 2: Plan localization to source.** We map each high-level plan to specific edit locations by aligning its objectives with neighborhoods of the code graph (modules/files) and extracting the relevant APIs and invariants from the leaf-level documentation cards. The result is a *granular plan*: an ordered set of diffs annotated with preflight checks (build, unit, flow-smoke tests), runtime probes (timing, design rule check (DRC), and routing metrics), and post-conditions tied to QoR deltas. The plan connects a literature-derived idea to verifiable changes in OpenROAD source and scripts.

**Stage 3: Autonomous execution with verification loops.** A repository-level coding agent, operating in a planner–executor architecture, applies the proposed diffs, compiles the modified code,


*ICCAD '26, San Jose, CA, USA*

ACM ISBN 979-8-4007-2873-0/2026/11
https://doi.org/10.1145/3831252.3834240

and runs instrumented OpenROAD flows on designated benchmarks. The agent iterates using self-measured signals (e.g., routed wirelength (rWL), worst negative slack (WNS), total negative slack (TNS), via count and power) together with guardrails such as revert-on-regression and bisect-on-failure. It uses tool-use patterns (code search, syntax trees, build logs) similar to SWE-agent [7], and can when applicable invoke tool APIs learned from examples (cf. Toolformer) [8]. Failures (compile errors, runtime crashes, QoR regressions) are converted into counterexamples and fed back into the granular plan for repair.

**Stage 4: Formal diff checking.** For selected diffs that admit lightweight formal contracts, we attach a Lean theorem encoding the intended invariant, use an automated prover (Aristotle [9]) to generate the proof, and validate it with a checker (AXLE [10]). Proof failures trigger diff revision. Section 3.5 details the mechanism and an OpenROAD case study.

**Contributions.** We introduce **AuDoPEDA**, a system for autonomous EDA code improvement. Our contributions are:

- **An autonomous system** that modifies the source code of a production EDA toolchain and improves QoR under closed-loop feedback.
- **A task suite and evaluation protocol** for autonomous EDA code improvement, with fixed OpenROAD/OpenROAD-flow-scripts (ORFS) baselines, three public process design kits (PDKs) (ASAP7, SKY130HD, Nangate45) and QoR-based acceptance gates scoped to core physical-design modules (detailed placement (DPL), global placement (GPL), resizer (RSZ)).
- **End-to-end demonstration on OpenROAD**, where each optimization target is addressed by a single autonomously discovered repository-level diff — reused across multiple PDKs and designs rather than redesigned per benchmark — achieving routed wirelength reductions of up to 5.9%, effective clock period (ECP) reductions of up to 10.0%, and power reductions of up to 19.4%.
- **A demonstrated formal verification gate for diffs**, where applicable: the agent can emit Lean contracts and proof artifacts that are automatically repaired (Aristotle [9]) and checked (AXLE [10]), enabling proof-carrying patches and tighter safety loops.

## 2 Related Work

Our work intersects four research vectors: the open-source EDA ecosystem, autonomous agents for software engineering, advanced methods for code representation and contextualization, and the application of LLMs to hardware design automation.

**The Open-Source EDA Ecosystem.** The OpenROAD project provides an open platform for register-transfer level (RTL)-to-GDSII implementation, targeting 24-hour, fully automated flows [1, 11]. It integrates engines for global placement [12, 13], global and detailed routing [14–16], and static timing analysis [18], composed into reproducible pipelines by OpenROAD-flow-scripts (ORFS) [19–21]. Together they provide standardized metrics, verifiable outputs, and scriptable interfaces for closed-loop evaluation. The heterogeneous C++/Tcl/Python OpenROAD codebase is the setting we target with S0–S2.

**Autonomous Agents and Repository-Scale Coding.** Recent code LLMs handle localized coding tasks well [22–24]. For repository-level engineering — requiring navigation, dependency tracking, and test-based validation — benchmarks such as SWE-bench [25] and agent frameworks such as SWE-agent [7] and AutoCodeRover [26] provide baselines; SWE-EVO [27] extends these benchmarks to long-horizon evolution scenarios, while recent work [28] raises memorization concerns for SWE-bench evaluation. We reuse established planning and tool-use patterns from this line of work [6, 8, 29–31] but replace unit-test acceptance with physical-design QoR metrics.

**Code Representation and Contextualization.** Learned code representations [32, 33] and graph-based retrieval [5, 34] improve context selection over naive lexical search [4], and agentic RAG frameworks scale retrieval with hierarchical interfaces [35]. We build on this line of work for S0/S1 retrieval.

**LLMs in Electronic Design Automation.** LLMs are being used across EDA, from RTL generation and design assistance to physical-design optimization and verification [36–41, 54]. More recently, agentic AI for physical-design R&D has been surveyed [42]. Classical machine learning/reinforcement learning (RL) work has also improved flow tuning and placement [43, 44]. Our setting is different: instead of generating RTL or tuning existing knobs, we edit the implementation of core OpenROAD PD modules.

**Formal and Static Verification as Diff Gates.** Our S4 stage follows proof-carrying artifacts and sound static analysis: safety properties can be checked automatically at a module interface [45], abstract interpretation can enforce invariants at low human cost [46], and satisfiability-modulo-theories (SMT) solvers can prove first-order obligations [47]. Recent LLM-based provers such as Goedel-Code-Prover [48] and APOLLO [49] demonstrate that automated Lean proof search is becoming practical. We apply this style to EDA development by binding diffs to small reusable contracts checked in continuous integration (CI) and by treating proof/analyzer failures as counterexamples for plan repair.

Unlike prior work focused on RTL generation, scripting assistance or parameter tuning, AuDoPEDA targets *autonomous code modification within the core C++/Tcl PD stack itself*. It keeps documentation, planning, and QoR-gated execution in one loop so the agent can change algorithms rather than only scripts or settings. Flow-tuning methods such as Bayesian optimization and RL search a fixed knob space without modifying the underlying algorithms; an accepted diff, by contrast, modifies a decision point in the source code itself and is reused unchanged across its evaluated designs and PDKs. Relative to DRC-Coder [36] and similar LLM-scripting agents, the key distinction is that the agent modifies algorithms across multiple modules while repeatedly verifying through actual flow results that cross-module invariants are preserved. Compared to SWE-bench/SWE-agent baselines, EDA imposes longer feedback loops (full place-and-route flows take minutes to hours per evaluation), multi-language toolchains (C++/Tcl/Python/Verilog), and acceptance criteria defined by physical metrics rather than unit tests. These differences motivate the graph-structured documentation (S0), domain-tagged retrieval (S1), and QoR-gated execution (S3) that distinguish our pipeline from a generic SWE-agent deployment.

## 3 Methodology

Our system, AuDoPEDA, decomposes the task of autonomous EDA code contribution into four structured stages plus an optional formal verification gate: (S0) repository graphing and documentation bootstrap, (S1) literature-grounded planning, (S2) plan localization to code, (S3) autonomous execution with QoR feedback, and (S4) formal diff checking. Each stage consumes and produces versioned artifacts (graphs, cards, plans, diffs, metrics), allowing traceability and replay under a pinned evaluation setup. We target the OpenROAD stack [1, 2] and its components (e.g., RePlAce, FastRoute, TritonRoute, OpenSTA) [12, 14, 15, 18]. Figure 1 shows the overall pipeline.

## 3.1 S0: Repository Graphing and Documentation Bootstrap

**Goal.** To make a large, multi-language EDA codebase (like OpenROAD) understandable to an autonomous agent, we first construct a detailed map of the code—a property graph $G$—and generate machine-usable documentation cards. These artifacts capture the code's structure, interfaces, and invariants, enabling downstream planning.

**S0.1 Parsing and Graph Construction.** We analyze the OpenROAD repository, covering C/C++ (core engines), Tcl (scripts), Python (utilities), and Verilog (design inputs).

- **Language-Agnostic Parsing.** We use *tree-sitter* [3] to parse these languages into abstract syntax trees (ASTs). We incorporate build information (from CMake) to accurately resolve dependencies and include paths, ensuring the parser understands the code as the compiler does.
- **The Code Graph ($G$).** We translate the ASTs into a unified graph representation. Nodes in $G$ represent key code elements: Files, Declarations (types, functions, classes), Definitions (implementations), and Callsites.
- **Typed Edges.** We connect these nodes with typed edges that capture relationships, such as `calls`, `includes/imports`, and variable `bindings`.

**S0.2 Cross-Language Linking and Simplification.** A crucial step is linking the scripting interface (Tcl/Python) to the C++ core. We identify where Tcl commands are registered in C++ (such as `Tcl_CreateCommand` wrappers) and add `script_invokes` edges. This allows the agent to trace execution from a high-level script command down to the engine implementation.

To manage complexity, we simplify the graph:

- **Condensation.** We condense tightly coupled groups of functions (Strongly Connected Components) into single nodes, reducing overhead from internal utility calls.
- **Pruning and Filtering.** We prune less informative edges in dense areas and exclude non-essential code, such as third-party libraries and test-only files.

**S0.3 Docmaker Pipeline and Documentation Cards.** The "Docmaker" automatically generates documentation cards by traversing the graph $G$ bottom-up (from utilities to main entry points).

- **Evidence Extraction.** For each node, we extract key information: function signatures, default parameters, assertions, configuration flags (Tcl options), error messages, and neighborhood context.
- **LLM-Based Summarization.** A code-specialized LLM converts this evidence into a concise summary card with standardized sections: `Role`, `Owned State`, `Inputs/Outputs`, `Pre/Postconditions`, `Config Knobs`, and `Editing Hazards`.
- **Validation.** We validate the generated cards for accuracy. We check that referenced APIs exist in $G$. Further, the summary must match the code's type signatures. This ensures that the documentation is reliable for the agent.

**S0.4 Indexing and Retrieval.** To enable efficient search, we index cards and code snippets with Best Match 25 (BM25) sparse retrieval and modern dense embeddings. This is combined with structural information from the graph, prioritizing results that are contextually relevant [5]. The pipeline is incremental, updating only affected parts of the graph when the codebase changes.

In practice, the documentation set is refreshed incrementally: changed files trigger regeneration of only the impacted cards, neighborhood summaries, and retrieval manifests, rather than a full rebuild of the entire documentation tree. This keeps the documentation pass compatible with a frequently updated OpenROAD codebase and reduces the chance that stale summaries dominate retrieval.

## 3.2 S1: Literature-Grounded Planning (DSPy)

**Goal.** Generate high-level, executable research plans combining knowledge of OpenROAD (from S0) with EDA literature.

**Knowledge Sources.** We maintain two documentation sets for retrieval: $C_{\text{repo}}$ (the Docmaker cards from S0) and $C_{\text{lit}}$ (EDA papers, tutorials, and documentation covering placement, routing, timing, etc.). Documents are domain-tagged (e.g., `gp`, `sta`) for retrieval.

**DSPy-Programmed Planner.** We treat planning not as a single prompt, but as a declarative LLM program compiled using *DSPy* [6]. This approach provides structure and reliability. The program follows a Retrieve-Synthesize-Validate structure:

(1) **Retrieve.** Given an objective (e.g., *reduce routed wirelength*), the planner retrieves relevant information from both $C_{\text{repo}}$ and $C_{\text{lit}}$ [4]. A re-ranker ensures the selected documents offer diverse perspectives and align with the repository structure.
(2) **Synthesize.** The planner integrates this information into a *high-level plan* with fields such as `objective`, `hypotheses`, `interventions`, `likely_touched_files`, `metrics`, `risks`, and `validation_plan`. It also suggests potential code locations and APIs for the interventions.
(3) **Validate.** The plan is checked against predefined constraints (language-model (LM) assertions — programmatic checks that reject plans referencing nonexistent APIs, out-of-range parameters, or violated invariants) and the code graph $G$. This ensures that proposed interventions are feasible (parameters are within range, referenced APIs exist, core invariants are respected) before execution.

The planner is therefore not allowed to return free-form prose alone. Each accepted idea must name likely edit locations, measurable metrics, and rollback signals so that S2/S3 can execute it without manual intervention.

## 3.3 S2: Localization and Granular Plans

**Goal.** Translate the high-level plan from S1 into an ordered sequence of source edits and verification steps.

**Edit-Surface Localization.** We identify the specific locations (files, functions, scripts) in the codebase where changes are needed by mapping the interventions from S1 onto the code graph $G$. We aim to find the minimal set of nodes that cover the required APIs and passes, while also minimizing the impact scope — the potential impact on dependent modules, assessed using graph centrality and historical change frequency.

**Granular Plan Representation.** The output is a *granular plan* detailing the implementation sequence. Each step includes:

- $\Delta_i$: The intended code modification (diff intent).
- $\texttt{Pre}_i$: Pre-run checks (build health, unit tests, formatting).
- $\texttt{Run}_i$: The specific OpenROAD flow configuration to execute (design, PDK, parameters).
- $\texttt{Probe}_i$: The metrics to monitor (QoR counters, logs).
- $\texttt{Post}_i$: Acceptance criteria (QoR improvement targets) and rollback conditions if the change causes regressions.

This structure ensures that the plan is verifiable, deterministic, and safe to execute autonomously.

### 3.4 S3: Autonomous Execution with QoR Feedback

**Goal.** Execute the granular plan, apply code modifications, run EDA flows, and improve the results based on measured QoR.

**Agent Architecture and Tools.** We employ a repository-level coding agent structured with a planner–executor interface, similar to SWE-agent [7]. The executor uses a specialized tool palette for interacting with the EDA environment:

- `code_edit`: Apply structured edits directly at the AST level (using tree-sitter coordinates).
- `build`: Compile OpenROAD and check unit tests.
- `flow`: Run non-interactive OpenROAD flows up to specific checkpoints (placement, routing, or signoff static timing analysis (STA)).
- `measure`: Extract QoR metrics from tool reports (routed wirelength and DRCs from TritonRoute; WNS, TNS, and total power from OpenSTA).
- `rollback` / `bisect`: Revert changes or isolate regressions.

**Autonomy Telemetry and Replayability.** Beyond QoR metrics, each run emits a structured trace (model calls/tokens, candidate diffs, build and flow invocations, failure modes and rollback reasons). We version these traces alongside the accepted diff and QoR reports with logging; this supports replay and failure analysis.

**Run-Root Artifacts.** Each unattended execution produces a versioned run root containing the final decision, per-iteration diffs, verification outputs, metric deltas and preflight checks, so that failures are inspectable artifacts rather than transient terminal output.

**QoR Gating and Metric Model.** We evaluate candidate changes with a signed, baseline-normalized score $S = \sum_i \alpha_i I_i$, where $I_i = (m_i^{\text{base}} - m_i^{\text{cand}})/\max(|m_i^{\text{base}}|, \epsilon)$ for lower-is-better metrics (sign reversed otherwise). We give the target metric unit weight and use smaller, objective-specific weights for secondary timing and routing terms. The score is subordinate to the following *gates*: a change is immediately rejected if it introduces new DRCs, worsens WNS beyond a predefined threshold, or causes build/test failures. A candidate is accepted only if the composite score improves and all gates pass.

**Search and Iteration Policy.** The agent operates in a closed feedback loop. It alternates between (a) *local repair* (fixing build errors or test failures introduced by a diff) and (b) *QoR iterative improvement*. During iterative improvement, the agent proposes several diffs, evaluates them using fast proxy flows (place followed by global route), and promotes the most promising candidates to full signoff runs. The best non-worsening change is committed. Failures are fed back to the planner (S1/S2) to refine future attempts.

### 3.5 S4: Formal Diff Checking (Lean + Static Analysis)

**Goal.** Provide a *machine-checkable* safety layer for diffs beyond unit tests and flow metrics, suitable for the subset of EDA changes that admit lightweight formal contracts.

**Proof-Carrying Diffs.** For a candidate diff $\Delta$, the agent may emit a formal contract $\Phi(\Delta)$ as Lean source code: a `formal_statement` file that encodes a theorem signature with `sorry` placeholders (proof holes to be filled automatically) and a `content` file intended to prove it. We rely on (i) a prover API (Harmonic Aristotle [9]) to fill/repair proofs and (ii) a checker API (Axiom AXLE [10]) to validate that the proven theorem matches the statement, contains no `sorry`, and compiles in a fixed Lean environment. If the proof fails, the system treats this as a counterexample and triggers rollback/repair. We demonstrate S4 on an OpenROAD GPL case.

**OpenROAD Case Study.** We apply S4 to a real C++ diff in OpenROAD's timing-driven net weighting (`src/gpl/src/timingBase.cpp`). The patch enforces `net_weight_max_` $\geq 1$ and clamps the computed weight accordingly. We attach a Lean contract that proves the corresponding boundedness theorem under an idealized Real-valued model, and check it automatically with AXLE [10].

**Diff–Contract Binding.** To make $\Phi(\Delta)$ checkable in CI, we bind each patch to a small manifest that (i) pins the target repository snapshot, (ii) checks via `git apply –check` (or reverse-check if the patch is already present) that the patch matches that snapshot, and (iii) runs AXLE [10] on the associated statement/proof pair. This guards against mismatches where the proof and diff drift apart. We emphasize that the contract models only the relevant numerical invariant (e.g., clamping and capping) and not the full C++ semantics; nevertheless, it blocks a common class of "silent constraint weakening" edits (e.g., removing a cap or allowing negative weights).

**Lean Obligation Shape.** The obligations are intentionally lightweight and reusable. For example, the timing-weight boundedness theorem has the shape:

```
theorem timingWeight_bounds : 1 ≤ w ∧ w ≤ w_max
```

where $w$ is the clamped weight and $w_{\max}$ is the capped maximum; Aristotle [9] can synthesize short proofs for these templates, and AXLE [10] checks them in a pinned environment (no `sorry`).

**Contract Templates and Repair Loop.** We maintain a small library of reusable contract patterns (bounds, guards, monotone costs) parameterized by a few scalars (caps, thresholds, penalty weights). Given a diff, the agent instantiates a template and emits a statement with localized `sorry` goals. Aristotle [9] then attempts to synthesize missing proof terms or minimally refactor the statement by introducing an explicit precondition when needed (e.g., adding $lo \leq hi$ for a clamp contract). AXLE [10] provides the final trust anchor by checking the resulting Lean kernel proof under a pinned toolchain [50, 51].

**Scope and Replicability.** S4 covers narrow numerical invariants (bounds, guards, monotone costs); broader C++ hazards (undefined behavior, memory safety) require complementary static-analysis or SMT-backed gates [46, 47]. To confirm that a contract is not vacuously true, we optionally run *negative tests* (e.g., remove the clamp) and verify that the proof fails. The planner invokes S4 when a change can be modeled cleanly, and otherwise falls back to build/test + QoR gates. Aristotle is remote, while AXLE is accessed through its public API and client; local replication requires these services or Lean installation.

### 3.6 Evaluation Protocol and Implementation

**Experimental Setup.** We evaluate the system using standard open designs on public PDKs (ASAP7, SKY130HD, Nangate45) with a fixed OpenROAD snapshot. We use both short proxy flows for quick evaluation and full signoff flows for final validation. Builds are reproduced from a pinned OpenROAD/ORFS commit, and all artifacts (plans, diffs, evaluation summaries) are versioned. The repository [52] includes a script that rebuilds the evaluation directory layout, and source files for reported runs.

**Safety and Rollback.** The system employs multiple layers of safety guards. (i) *Static guards* in S1/S2 check API existence and respect invariants before execution. (ii) *Runtime guards* in S3 enforce build, test, DRC and timing gates. (iii) *Formal guards* in S4 validate numerical invariant contracts for applicable diffs and run static-analysis gates on C++ edit locations. Violations trigger an automatic rollback, and the failure is recorded as a counterexample to inform future planning attempts. We cache intermediate flow results (e.g., placed

| Objective | Cand. | Wins | Median | Mean (95% CI) |
|---|---|---|---|---|
| rWL | 36 | 33 | 2.35% | 2.65% ([2.03, 3.27]%) |
| ECP | 30 | 28 | 4.10% | 4.75% ([3.51, 5.99]%) |
| Power | 34 | 31 | 7.40% | 8.65% ([6.30, 11.00]%) |

**Table 1: Final-signoff audit by primary objective. Each row's median and mean (95% CI) use the improving candidates for that objective; 92 of 100 candidates improve.**

| Stage | Calls | Input | Output | Cost |
|---|---|---|---|---|
| S0 cards + validation | 760 | 20.50M | 2.02M | $42.50 |
| S1 planning | 54 | 2.80M | 0.36M | $7.20 |
| S2 localization | 61 | 1.90M | 0.28M | $5.00 |
| S3 generation + repair | 210 | 7.40M | 1.38M | $19.70 |
| S4 proof work | 23 | 0.62M | 0.09M | $1.80 |
| Reporting | 18 | 0.42M | 0.07M | $0.80 |
| Total | 1126 | 33.64M | 4.20M | $77.00 |

**Table 2: Audited LLM calls, tokens, and estimated cost by stage.**

design exchange format (DEF)) to accelerate iterative improvement loops.

## 4 Experiments

We evaluate AuDoPEDA on three QoR optimization targets — routed wirelength, effective clock period, and total power — scoped to modifications in **DPL, GPL, RSZ** (Detailed Placement, Global Placement, Resizer) of OpenROAD. Unless labeled otherwise, runs use OpenAI Codex/GPT-5.2 on a 112-vCPU x86 VM with 220 GB RAM. We use ORFS (commit hash: `93c42b`) and OpenROAD (commit hash: `7bc521`). Every candidate has a separate worktree and run root containing its plan, patch, build and flow logs, extracted metrics, and accept/reject decision.

A cold S0 pass takes about 4.6 hours: 22 minutes to construct the code graph, 3.6 hours to generate documentation cards, 31 minutes to validate the cards, and 14 minutes to build the index. Per objective, S1 planning takes 12–18 minutes and S2 localization takes 8–15 minutes. Editing takes 5–25 minutes. OpenROAD builds take 38–45 minutes from a cold state and 6–18 minutes incrementally. A proxy flow takes 35–90 minutes per design; full signoff takes 1.7–5.8 hours. Physical-design runs account for most of the wall-clock time.

Table 1 reports the final-signoff audit of 100 candidate diffs, where 92 of the 100 candidates improve their primary objective.[1] The three diffs analyzed in detail below come from this set. Ablating S0/S1/S2/S3/S4 leaves 44/26/37/54/41 valid diffs, respectively. In the 1-ps timing sweep ($n = 404$), mean ECP improvement is 4.80%±2.90% (95% CI [4.52, 5.08]%). The LLM ledger records 1,126 calls, 33.64M input tokens, and 4.20M output tokens, at an estimated total cost of US$77. These results and the source codes are public [52].

In total, each objective search takes 34–45 hours. The 0.19-second Lean check contributes negligibly to this runtime.

### 4.1 Improvement of Wirelength

To reduce final routed wirelength, the agent restructures the detailed-placement global swapper into a two-pass engine. First, a *profiling pass* performs cell swaps based solely on HPWL reduction without considering congestion; this gives an estimated upper bound on achievable HPWL reduction for the given design. Then, a congestion-aware *optimization pass* uses this estimate as a reference to jointly manage congestion and wirelength, allowing up to a budgeted limit of HPWL increase when a move improves routability. Algorithm 1 gives details of the two-pass approach. We evaluate on `aes`, `ibex`, and `jpeg` across ASAP7, SKY130HD, and Nangate45, plus four macro-heavy Nangate45 designs (`ariane133`, `ariane136`, `bp_fe`, `swerv_wrapper`) commonly used in prior flow studies [43].

**Algorithm 1:** Two-pass congestion-aware global swap

```
// Pass 1: profiling
H_opt ← GlobalSwap(HPWL-only)
B ← H_opt · (1+ϵ)
RestoreInitialPlacement()
// Pass 2: congestion-aware optimization
U ← ComputeUtilMap(area, pins)
w_c ← CalibrateWeight(trial swaps, U)
for each budget stage b_k do
    B ← H_opt · b_k
    foreach swap (c_i, s_j) do
        Δ_h ← HpwlDelta(c_i, s_j)
        Δ_g ← CongRelief(c_i, s_j, U)
        p ← Δ_h + w_c·Δ_g
        if p > 0 and HPWL ≤ B then
            Accept(c_i, s_j)
            UpdateUtilMap(U)
```

At the engine's core is a cost function that balances the placement wirelength estimate against a site-level utilization density metric (i.e., a per-region measure of how crowded each placement site is). This metric combines cell area and pin count within each placement region: the area component identifies physically congested areas where routing resources are scarce, while the pin component captures regions with high routing demand where many nets converge. Broadly, the swapper penalizes moves that would exacerbate congestion and disperses cells from high-density regions, thus preserving track availability and enabling more direct net routing paths. The agent's code modification is localized to the detailed-global swap generator and its acceptance rule: legalization, row construction, filler insertion, and downstream routing engines are unchanged.

In the optimization pass, generation of cell swaps mixes deterministic HPWL-improving proposals with stochastic exploration. An adaptive weight is estimated from trial swaps to scale the density term into the same units as HPWL. As noted, this pass accepts a move only when expected density relief justifies any HPWL increase. The optimization iterates over progressively tighter budget stages, each allowing a smaller HPWL increase relative to a profiled optimum.

Table 3 shows the flow configurations that we use to evaluate the code change. The utilization values are "high-stress": they are obtained by incrementing core utilization upwards in steps of five percentage points until routability failure occurs.[2]

Table 4 summarizes the results.[3] We apply the same autonomous diff unchanged across all platforms and designs. In the runs underlying Table 4, we also checked ECP, power, instance area, and instance

[1] Eight candidates fall outside the 92 improvements: two fail build or proxy evaluation, one lacks a usable signoff metric, two fail a secondary-objective check, and three pass signoff without reaching the primary-objective improvement threshold. Overall, 95/100 are non-worsening at full signoff and 92/100 strictly improve.

[2] The headings name ORFS parameters: Util is `CORE_UTIL`, LB add-on is `PLACEMENT_LB_ADDON`, Aspect is `CORE_ASPECT_RATIO`, and Margin is `CORE_MARGIN`. DPO enabled means `ENABLE_DPO=1`; EQY disabled means `EQUIVALENCE_CHECK=0`. We leave `CORE_AREA` and `DIE_AREA` undefined so that `CORE_UTIL` determines the floorplan.

[3] N/A means that the baseline flow did not produce a routed solution at this utilization.

| PDK | Design | Util | LB add-on | Aspect (Margin) |
|---|---|---|---|---|
| ASAP7 | aes | 75% | 0.2 | – |
| ASAP7 | ibex | 70% | 0.2 | – |
| ASAP7 | jpeg | 70% | 0.2 | – |
| SKY130HD | aes | 30% | 0.2 | – |
| SKY130HD | ibex | 50% | 0.2 | – |
| SKY130HD | jpeg | 60% | 0.2 | – |
| Nangate45 | aes | 85% | 0.2 | – |
| Nangate45 | ibex | 30% | 0.2 | – |
| Nangate45 | jpeg | 30% | 0.2 | – |
| Nangate45 | bp_fe | 30% | 0.11 | – |
| Nangate45 | ariane133 | 30% | – | 1(5) |
| Nangate45 | ariane136 | 30% | – | – |
| Nangate45 | swerv_wrapper | 30% | 0.08 | 1(5) |

**Table 3: Flow configurations for wirelength improvement testing, all with DPO enabled and EQY disabled.**

| PDK | Design | Baseline rWL | Our rWL | ΔrWL (%) |
|---|---|---|---|---|
| ASAP7 | aes | 64,640 | 62,710 | −2.99 |
| ASAP7 | ibex | 80,402 | 80,823 | +0.52 |
| ASAP7 | jpeg | 154,484 | 152,232 | −1.46 |
| SKY130HD | aes | 659,778 | 633,899 | −3.92 |
| SKY130HD | ibex | 646,855 | 643,006 | −0.60 |
| SKY130HD | jpeg | N/A | 1,201,778 | N/A |
| Nangate45 | aes | 230,044 | 217,415 | −5.49 |
| Nangate45 | ibex | 248,641 | 248,429 | −0.09 |
| Nangate45 | jpeg | 565,979 | 554,902 | −1.96 |
| Nangate45 | bp_fe | 1,603,884 | 1,634,916 | +1.94 |
| Nangate45 | ariane133 | 7,831,361 | 7,523,708 | −3.93 |
| Nangate45 | ariane136 | 7,986,048 | 7,509,944 | −5.96 |
| Nangate45 | swerv_wrapper | 4,310,916 | 4,239,837 | −1.65 |

**Table 4: Routed wirelength (rWL, in units of $\mu$m) outcomes from wirelength improvement testing.**

count; no systematic regression in these secondary metrics was observed, even though they are not the primary optimization target in this study. **Two cases show slight regressions**: ibex on ASAP7 (+0.52%) and bp_fe on Nangate45 (+1.94%). In both of these cases, baseline congestion is low enough that the density-relief mechanism finds few beneficial swap candidates, and the profiling overhead slightly perturbs placement quality. Conversely, SKY130HD jpeg is marked N/A because the baseline flow fails to produce a routed solution at 60% utilization; the modified flow completes routing successfully, indicating that the congestion-aware swap policy extends the routability envelope. Last, this autonomously generated detailed-placement change was submitted upstream to OpenROAD as pull request #9038, and subsequently merged into the master branch [55] after passing human maintainer review.

## 4.2 Better ECP at Aggressive Timing

In a second experiment, we give the system the docs of **GPL** (Global Placement) and **RSZ** (Resizer), and seek a diff that lowers ECP when target clock period (TCP) is tightened to 0.85× the default in ORFS. The effective clock period ECP = TCP − WNS is the fastest clock at which the chip meets timing: it equals TCP when all constraints are met, and grows beyond TCP when timing violations exist. The accepted diff modifies two existing documented OpenROAD modules: GPL's timing-driven reweighting path (via TimingBase, which updates GNet::timingWeight_ at overflow-triggered checkpoints)

**Algorithm 2:** Timing-weight shaping (GPL + RSZ)

```
// GPL: per-net weight
s_ref ← SetSlackRef(zero_ref)
N ← ApplyCoverage(worst nets, k%)
ℓ̄ ← AvgBboxLength(N)
foreach net ∈ N with slack s < s_ref do
    v ← NormalizeSlack(s, s_ref, s_min)
    v ← v^γ
    v ← ApplyLengthFactor(v, ℓ, ℓ̄, α)
    weight ← Clamp(1+(w_max−1) v, 1, w_max)
// RSZ: convergence guard
m ← LoadMaxDegradingPasses()
for each repair pass do
    if WNS ↓ and TNS ↓ then count++
    if count > m then RollbackToCheckpoint()
```

and RSZ's repair_timing loop. Rather than globally increasing timing pressure, the agent applies four implementation rules:

- **Path-scoped weighting with bounded multipliers.** The placer still computes slack-based net weights, but the diff caps the maximum multiplier, can set the slack reference to zero, and restricts aggressive reweighting to nets with the worst slack so that timing pressure remains concentrated on critical nets.
- **Length- and congestion-aware emphasis.** Beyond slack, the diff adds a net-length term and an overflow-aware congestion check: if a critical net's bounding box crosses a high-congestion bin, its timing multiplier is damped toward 1.0 rather than forcing the placer to worsen routability.
- **Resizer guardrails.** In RSZ, the flow rejects moves that consume too much positive slack on non-critical endpoints, enforces a hold-slack floor (minimum timing margin for hold constraints), and prefers more disciplined high-fanout handling so setup repair does not create new downstream problems.
- **Convergence controls.** The repair loop bounds the number of slack-degrading passes and exposes the parasitics-analysis choice (selection between faster versus more accurate wire delay models) as an explicit runtime/fidelity knob, making the search more robust under aggressive TCP tightening.

In OpenROAD terms, the GPL portion changes which nets enter the worst-net set and how their respective multipliers are computed, while the RSZ portion changes which repair candidates remain admissible once setup improvement begins to threaten hold or positive-slack guardbands. Intuitively, the placer creates a better geometric starting point for critical nets, and RSZ then spends less effort undoing routability damage or reprising near-critical endpoints. Algorithm 2 summarizes the GPL weight computation; the RSZ side bounds the number of slack-degrading repair passes via a configurable limit,
and the combined diff reclaims slack under a tighter clock while preserving routability and hold margins. We perform evaluation on larger Nangate45 circuits (details given in Table 5).[4] For these circuits, the target clock period is reduced by 15% (50% in the case of ariane136) from the default value in ORFS. As shown in Table 6, the same GPL+RSZ diff achieves ECP reductions relative to baseline OpenROAD.

**Diff reuse and module interaction.** The Table 6 results are from a single GPL+RSZ diff that is reused across all four circuits. The gains also depend on interaction between the two modules, i.e., both

[4] In the ORFS benchmark configuration, one macro is omitted from the ariane133 netlist.

| Design | #Cells | #Macros | #Nets | #Pins |
|---|---|---|---|---|
| ariane133 | 184314 | 132 | 195662 | 620474 |
| ariane136 | 194547 | 136 | 205959 | 643350 |
| bp_fe | 38924 | 11 | 41418 | 114292 |
| swerv_wrapper | 107466 | 28 | 113694 | 358017 |

**Table 5: Attributes of larger Nangate45 circuits.**

| Design | TCP | Baseline ECP | Our ECP | ΔECP |
|---|---|---|---|---|
| ariane133 | 3.4 | 3.59 | 3.43 | -4.6% |
| ariane136 | 3.0 | 3.78 | 3.41 | -10.0% |
| bp_fe | 1.53 | 1.71 | 1.65 | -3.4% |
| swerv_wrapper | 1.7 | 2.19 | 2.16 | -1.4% |

**Table 6: Timing results in Nangate45 (TCP/ECP in ns).**

placement-time reweighting and repair-time guardrails contribute to alignment of timing pressure, routability protection, and post-placement repair within the RTL-to-GDSII pipeline. A census of public macro-containing ORFS designs at the stated hash found two more runnable, non-duplicate families. The same diff improves ECP by 2.01% on Nangate45 `mempool_group` and by 2.86% on ASAP7 `riscv32i` at 70% TCP (1.61% at 85%). The remaining macro cases require proprietary PDKs that were unavailable for this study. The repository [52] includes this census.

## 4.3 Improvement of Power

Our third experiment gives RSZ documentation and asks the system to reduce power by modifying the Resizer module in OpenROAD. The baseline OpenROAD flow uses its power-recovery mode with `RECOVER_POWER=100`, i.e., we measure our diff against an existing power-oriented RSZ flow rather than against timing repair that has power recovery disabled. The accepted diff replaces path-local recovery with an instance-ranked flow. It scores cells by power contribution and timing margin, then applies staged buffer removal, downsizing, and swaps to higher-threshold-voltage (higher-VT) cells to reduce leakage under timing guards. This differs from baseline `recover_power`, which selects positive-slack paths and applies local resizing. By moving the selection unit from path to instance and broadening the repair sequence, the diff can touch timing-safe high-power cells invisible to a path-localized heuristic. Algorithm 3 summarizes the approach.

**Algorithm 3:** Instance-ranked power recovery

$w_f \leftarrow$ ComputeWnsFloor(WNS, recover%)
corner $\leftarrow$ SelectPowerCorner()
$P_0 \leftarrow$ MeasureTotalPower(corner)
$C \leftarrow$ CollectCandidates(non-clock, slack $>$ margin)
**foreach** $i \in C$ **do**
  score($i$) $\leftarrow$ PowerScore(power($i$), headroom($i$))
SortDesc($C$, score)
**foreach** top $k$% of $C$ **do**
  **if** $i$ is buffer **then** TryBufferRemoval($i$)
  TryDownsize($i$)
  TryVtSwap($i$, lower leakage)
  **if** WNS $< w_f$ **then** Rollback($i$)

Table 7 shows that lower power is achieved for all nine cases studied. We obtain each value from finish-stage OpenROAD `report_power_metric`, which sums internal, switching, and leakage power; every design uses `RECOVER_POWER=100`. (We note that prior work found reasonable correlation between OpenROAD/OpenSTA power and timing analyses and commercial tools [53].) The largest reductions are on `ibex`: 18.7% with ASAP7 and 19.4% with SKY130HD, accompanied by 2.6% and 4.4% ECP overhead. Across the nine rows, the instance-ranked sequence trades lower power for 1.3–9.8% higher ECP. The largest overhead occurs on ASAP7 `aes`, where aggressive downsizing trades timing margin for power reduction; in practice, the designer can tune this tradeoff by adjusting the timing guard threshold in the staged recovery flow. We evaluated this tradeoff at 27 public design/library points by sweeping baseline `RECOVER_POWER=100` and the proposed recovery; these power–ECP points extend the measured frontier for the tested settings. Power falls at 26/27 points (median 6.525%). At matched ECP (±0.5%), median power is 3.4% lower. With a 2% (resp. 5%) limit on ECP increase, median power is 6.2% (resp. 9.1%) lower.

## 4.4 Formal Gate Micro-Case Study: OpenROAD Timing-Weight Clamp

We evaluate S4 on a real OpenROAD diff that hardens timing-driven net weighting by enforcing `net_weight_max_` $\geq 1$ and clamping the computed weight accordingly (`src/gpl/src/timingBase.cpp`). This micro-case is a demonstration of the gate itself, not the accept/reject mechanism behind Tables 4–7. We bind the patch to a *proof-carrying manifest* with four pinned components: target repository snapshot, patch file, Lean statement, and Lean proof. The `verify-manifest` check first runs `git apply -check` (or reverse-check if the patch is already present), then calls AXLE's `verify_proof` endpoint in a pinned Lean environment on the statement/proof pair. The statement decomposes the obligation into short lemmas (`netWeightMax_ge_one`, `clamp_bounds`, `timingWeight_bounds`) over an idealized real-valued model. The contract is tied to the diff: if a future edit removes the clamp or permits weights below 1, the proof fails and the planner can restore an explicit cap instead of waiting for a QoR regression. The C++ patch is small. The setter enforces a 1.0 lower bound on `net_weight_max_`; the weight computation introduces `raw_weight` before its final `std::clamp`. The Lean model captures that lower bound and final clamp, but does not model full C++ semantics.

For the above-described patch, the diff is 29 lines; the Lean statement and proof are 34 and 40 lines; and the end-to-end `verify-manifest` check runs in 0.19 seconds (including the remote AXLE [10] call). We note that this example application of S4 proves neither the QoR tables nor full C++ semantics. Our repository also provides checked Lean modules for the three reported edit families: WL budget/profit bounds (68 theorems), ECP weight/repair guards (50), and power timing/monotonicity guards (49), all with zero `sorry`. In total, the static S4 layer scanned all 100 candidates. Of the 71 candidates with domain-specific obligations, three have complete Lean modules [52].

| PDK | Design | Baseline Pwr | Our Pwr | ΔPwr (%) | ΔECP (%) |
|---|---|---|---|---|---|
| ASAP7 | aes | 153.74 | 150.75 | −1.947 | 9.836 |
| ASAP7 | ibex | 58.10 | 47.24 | −18.703 | 2.606 |
| ASAP7 | jpeg | 119.70 | 117.48 | −1.853 | 2.880 |
| SKY130HD | aes | 456.94 | 437.55 | −4.245 | 3.529 |
| SKY130HD | ibex | 93.20 | 75.14 | −19.383 | 4.433 |
| SKY130HD | jpeg | 485.90 | 428.01 | −11.914 | 1.786 |
| Nangate45 | aes | 385.29 | 373.67 | −3.017 | 1.339 |
| Nangate45 | ibex | 95.98 | 89.37 | −6.883 | 3.644 |
| Nangate45 | jpeg | 499.22 | 454.71 | −8.915 | 5.647 |

**Table 7: Power outcomes (Pwr = total power, in mW) from power improvement testing.**

| PDK | Design | Baseline | Tuned | Source diff |
|---|---|---|---|---|
| SKY130HD | aes | 660096 | 648242 | 633194 |
| Nangate45 | aes | 230044 | 222704 | 216577 |
| ASAP7 | jpeg | 154484 | 154175 | 152232 |

**Table 8: Matched routed-wirelength comparison (lower is better).**

| Early-stage Codex/GPT-5.2 outcome | Count |
|---|---|
| Generated candidates | 100 |
| CI/build clean | 63 |
| Proxy-valid | 41 |
| Promoted to full signoff | 38 |
| Full-signoff non-worsening | 34 |
| Full-signoff improving | 30 |

**Table 9: Proxy-to-signoff funnel. This early-stage cohort is distinct from the final-signoff audit in Table 1.**

## 5 Discussion

In this section, we examine characteristics shared by the reported diffs, compare source editing with knob tuning, report outcomes of search ablations, and discuss limitations of S4 and replay.

### 5.1 Matched Tuning Baseline and Proxy Audit

Table 8 shows that with the same wall-clock and compute budget, the OR-AutoTuner-style search lowers rWL, but the source diff is 2.11% better on average. Here, the comparison is only with respect to rWL evaluation, and there is no knob tuning on top of a source change. We emphasize that each autotuning result is specific to a single design, PDK and objective, whereas all three source results use the same diff.

In a separate Codex/GPT-5.2 run, Table 9 shows that 38 of 100 generated candidates passed CI and proxy gates and went to full signoff. Of these, 34 were non-worsening and 30 improved, with proxy-to-final correlations of Spearman $\rho = 0.86$ and Kendall $\tau = 0.82$. (Specifically: these correlations use candidates with both proxy and signoff metrics; they capture ranking agreement, but not error calibration.) The other four candidates either regressed, failed a required check, or failed during signoff. Hence, a proxy result does not decide final acceptance. With the same harness, Claude Opus/Sonnet 4.6 sent 30 candidates to signoff, of which 16 were non-worsening, compared with 34 of 38 for Codex/GPT-5.2. We therefore see that the choice of coding agent affects the signoff yield.

### 5.2 Context and Search Ablations

With repository+literature retrieval, 92/100 candidates improve at final signoff. This count changes to 71 with repository-only retrieval, 39 with literature-only retrieval, 26 without retrieval, 55 with one monolithic long-context prompt, and 48 with a static structured context. These are end-to-end ablations, so the count differences do not isolate the effects of single components. All counts use the same final-signoff acceptance rule. Repository-only still retrieves current code dynamically, while static context uses fixed file lists, dependency hubs, run instructions, metrics, and guards.

For the WL/ECP/PWR searches, the agent generated 24/19/22 candidates and sent 5/4/5 to full signoff; the reported diff first appeared at iteration 22/17/20. The three searches used 38/30/34 compile attempts and 10/8/9 proxy-flow runs. Every candidate has a separate worktree and run root, and candidate patches are never combined.

### 5.3 Diff Characteristics Across Tasks

In our studies, each of the three code diffs is applied unchanged across its evaluated designs and PDKs. Each diff acts on a narrow mechanism that the system identified through the documentation, i.e., DPL's detailed-global swap acceptance rule, GPL's worst-net weighting with RSZ admissibility checks, and RSZ's power-recovery sequence. Each diff is concise enough for direct inspection, but it changes a decision point that the downstream flow visits many times. This explains how a single repository-level diff can affect flow-level outcomes across many instances without benchmark-specific code generation.

Importantly, AuDoPEDA produces diffs that are localized in their code-change scope. In every case, most changed lines lie in one core file (Table 10); the remaining files are headers, Tcl/SWIG bindings, or build-system updates. The ECP diff alone spans two modules. Because the rest of the RTL-to-GDS flow is intact, experimental comparisons and observed performance improvements indeed isolate repository-level algorithm edits from flow rewrites.

| Diff | Module(s) | Files | Lines ± | Core file (% of change) |
|---|---|---|---|---|
| WL | DPL | 11 | +1018/−43 | `detailed_global.cxx` (54%) |
| ECP | GPL+RSZ | 11 | +329/−25 | `timingBase.cpp` (46%) |
| PWR | RSZ | 9 | +1076/−388 | `RecoverPower.cc` (90%) |

**Table 10: Diff summary: files and line changes.**

### 5.4 Limitations

The experiments cover source changes on fixed public flows. They do not compare AuDoPEDA with human experts, a commercial tool, or a proprietary PDK. The matched OR-AutoTuner experiment covers routed wirelength only, and the ECP study is restricted to public macro-containing designs that run in the stated ORFS flow. Every generated patch is a standard Git diff that a maintainer can inspect, modify, or reject.

The results are tied to OpenROAD commit `7bc521` and ORFS commit `93c42b`. A manifest detects a patch that no longer applies or a proof that no longer checks, but it cannot predict QoR after a rebase. A new source snapshot, PDK, or design must be run through proxy and signoff again. Code generation is stochastic; the archived evaluation inputs and outputs make the measured runs auditable. The Lean contracts cover only the stated numerical invariants. Memory safety, undefined behavior, maintainability, and complete C++ correctness need other analyses.

## 6 Conclusions

With AuDoPEDA, we demonstrate that repository-level EDA code modification can be driven by documentation, structured planning, and QoR-gated execution. On OpenROAD, one diff per task reduces routed wirelength by up to 5.9%, effective clock period by up to 10.0%, and power by up to 19.4% across multiple designs and PDKs. In our 100-candidate audit, a candidate is considered a "success" only if it builds successfully, passes the required checks, and improves the design all the way through final signoff. By this standard, 92 of the 100 audited candidates succeed. Furthermore, one of these diffs, the DPL wirelength change described in Section 4.1, was merged into the OpenROAD master branch through pull request #9038 [55] after passing human maintainer review.

Our GitHub repository [52] and Zenodo snapshot contain the diffs, plans, proofs, and evaluation summaries that we report.

## Acknowledgments

This work is partially supported by the Samsung AI Center. OpenAI Codex/GPT-5.2 generated the patches evaluated in this paper. Claude Opus/Sonnet 4.6 and Gemini 2.5 Pro were used during system development and for manuscript editing.

## References


[1] The OpenROAD Project. https://openroad.org.

[2] The OpenROAD Project, "OpenROAD source repository," commit hash: 7bc521f36a34c986885473856e9f5b464093e38a, accessed Aug. 23, 2026. https://github.com/The-OpenROAD-Project/OpenROAD.

[3] M. Brunsfeld, "tree-sitter: An Incremental Parsing System for Programming Tools", 2018. https://tree-sitter.github.io.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 9459–9474.

[5] D. Sounthiraraj, J. Hancock, Y. Kortam, A. Javvaji, P. Singh and S. Shankar, "CodeCraft: Hierarchical Graph-Based Code Summarization for Enhanced Context Retrieval", arXiv:2504.08975, pp. 1–12, 2025.

[6] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq et al., "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines", *Proc. International Conference on Learning Representations (ICLR)*, 2024, pp. 1–32.

[7] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering", *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2024, pp. 50528–50652.

[8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda et al., "Toolformer: Language Models Can Teach Themselves to Use Tools", *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023, pp. 68539–68551.

[9] T. Achim, A. Best, A. Bietti, K. Der, M. Fédérico, S. Gukov, D. Halpern-Leistner et al., "Aristotle: IMO-Level Automated Theorem Proving", arXiv:2510.01346, pp. 1–23, 2025.

[10] J. Xin, A. Schneidman, C. Cummins, K. Ram, S. Ganesh and J. Limperg, "AXLE: A Cloud Infrastructure for Lean 4 Theorem Proving Utilities", arXiv:2606.26442, pp. 1–15, 2026.

[11] T. Ajayi, V. A. Chhabria, M. Fogaça, S. Hashemi, A. Hosny, A. B. Kahng, M. Kim et al., "Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project", *Proc. ACM/IEEE Design Automation Conference (DAC)*, 2019, pp. 76:1–76:4.

[12] C.-K. Cheng, A. B. Kahng, I. Kang and L. Wang, "RePlAce: Advancing Solution Quality and Routability Validation in Global Placement", *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 38, no. 9, pp. 1717–1730, 2019.

[13] Y. Lin, S. Dhar, W. Li, H. Ren, B. Khailany and D. Z. Pan, "DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement", *Proc. ACM/IEEE Design Automation Conference (DAC)*, 2019, pp. 1–6.

[14] A. B. Kahng, L. Wang and B. Xu, "TritonRoute: The Open-Source Detailed Router", *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 40, no. 3, pp. 547–559, 2021.

[15] M. Pan and C. Chu, "FastRoute: A Step to Integrate Global Routing into Placement", *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2006, pp. 464–471.

[16] M. Pan, Y. Xu, Y. Zhang and C. Chu, "FastRoute: An Efficient and High-Quality Global Router", *VLSI Design*, vol. 2012, Article ID 608362, 2012.

[17] A. Ghose, A. B. Kahng, S. Kundu and Z. Wang, "ORFS-Agent: Tool-Using Agents for Chip Design Optimization", *Proc. ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*, 2025, pp. 1–13, doi: 10.1109/MLCAD65511.2025.11189204.

[18] OpenSTA: Open-Source Static Timing Analyzer. https://github.com/The-OpenROAD-Project/OpenSTA.

[19] The OpenROAD Project, "OpenROAD-flow-scripts source repository," commit hash: 93c42b2e6877550589af655e0eb299038426e915, accessed Aug. 23, 2026. https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts.

[20] A. A. Ghazy and M. Shalan, "OpenLANE: The Open-Source Digital ASIC Implementation Flow", Article No. 21, *Proc. Workshop on Open-Source EDA Technology (WOSET)*, 2020.

[21] M. Shalan and T. Edwards, "Building OpenLANE: A 130nm OpenROAD-Based Tapeout-Proven Flow", *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2020, pp. 1–6.

[22] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards et al., "Evaluating Large Language Models Trained on Code", arXiv:2107.03374, pp. 1–35, 2021.

[23] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi et al., "Code Llama: Open Foundation Models for Code", arXiv:2308.12950, pp. 1–48, 2023.

[24] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li and S. C. H. Hoi, "CodeT5+: Open Code Large Language Models for Code Understanding and Generation", *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 1069–1088.

[25] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?", *Proc. International Conference on Learning Representations (ICLR)*, 2024, pp. 1–52.

[26] Y. Zhang, H. Ruan, Z. Fan and A. Roychoudhury, "AutoCodeRover: Autonomous Program Improvement", *Proc. ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 1592–1604.

[27] T. Le, M. V. T. Thai, D. N. Manh, H. P. Nhat and N. D. Q. Bui, "SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios", arXiv:2512.18470, pp. 1–30, 2025.

[28] T. Prathifkumar, N. S. Mathews and M. Nagappan, "Does SWE-Bench-Verified Test Agent Ability or Model Memory?", arXiv:2512.10218, pp. 1–4, 2025.

[29] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning", *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023, pp. 8634–8652.

[30] A. Singhvi, M. Shetty, S. Tan, C. Potts, K. Sen, M. Zaharia and O. Khattab, "DSPy Assertions: Computational Constraints for Self-Refining Language Model Pipelines", arXiv:2312.13382, pp. 1–17, 2023.

[31] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models", *Proc. International Conference on Learning Representations (ICLR)*, 2023, pp. 1–33.

[32] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages", *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 1536–1547.

[33] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou et al., "GraphCodeBERT: Pre-Training Code Representations with Data Flow", *Proc. International Conference on Learning Representations (ICLR)*, 2021, pp. 1–18.

[34] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao et al., "RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation", *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 2471–2484.

[35] M. Du, B. Xu, C. Zhu, S. Wang, P. Wang, X. Wang and Z. Mao, "A-RAG: Scaling Agentic Retrieval-Augmented Generation via Hierarchical Retrieval Interfaces", arXiv:2602.03442, pp. 1–18, 2026.

[36] C.-C. Chang, C.-T. Ho, Y. Li, Y. Chen and H. Ren, "DRC-Coder: Automated DRC Checker Code Generation Using LLM Autonomous Agent", *Proc. ACM/IEEE International Symposium on Physical Design (ISPD)*, 2025, pp. 143–151.

[37] Z. He and B. Yu, "Large Language Models for EDA: Future or Mirage?", *Proc. ACM/IEEE International Symposium on Physical Design (ISPD)*, 2024, pp. 65–66.

[38] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben et al., "ChipNeMo: Domain-Adapted LLMs for Chip Design", arXiv:2311.00176, pp. 1–23, 2023.

[39] S. Liu, W. Fang, Y. Lu, J. Wang, Q. Zhang, H. Zhang and Z. Xie, "RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique", *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 44, no. 4, pp. 1448–1461, 2025.

[40] B.-Y. Wu, U. Sharma, S. R. D. Kankipati, A. Yadav, B. K. George, S. R. Guntupalli, A. Rovinski et al., "EDA Corpus: A Large Language Model Dataset for Enhanced Interaction with OpenROAD", arXiv:2405.06676, pp. 1–5, 2024.

[41] J. Pan, G. Zhou, C.-C. Chang, I. Jacobson, J. Hu and Y. Chen, "A Survey of Research in Large Language Models for Electronic Design Automation", *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 30, no. 3, pp. 1–21, 2025.

[42] A. Ghose, A. B. Kahng, S. Kundu and B. Pramanik, "Agentic AI for Physical Design R&D: Status and Prospects", *Proc. ACM/IEEE International Symposium on Physical Design (ISPD)*, 2026, pp. 133–141.

[43] J. Jung, A. B. Kahng, S. Kim and R. Varadarajan, "METRICS2.1 and Flow Tuning in the IEEE CEDA Robust Design Flow and OpenROAD", *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2021, pp. 1–9.

[44] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. Songhori, S. Wang, Y.-J. Lee et al., "A Graph Placement Methodology for Fast Chip Design", *Nature*, vol. 594, no. 7862, pp. 207–212, 2021.

[45] G. C. Necula, "Proof-Carrying Code", *Proc. ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1997, pp. 106–119.

[46] P. Cousot and R. Cousot, "Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints", *Proc. ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 1977, pp. 238–252.

[47] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver", *Proc. International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008, pp. 337–340.

[48] Z. Li, Z. Yang, D. He, H. Zhao, A. Zhao, S. Tang, K. Yang et al., "Goedel-CodeProver: Hierarchical Proof Search for Open State-of-the-Art Code Verification", *Proc. Conference on Language Modeling (COLM)*, 2026, pp. 1–27.

[49] A. Ospanov, F. Farnia and R. Mohit, "APOLLO: Automated LLM and Lean Collaboration for Advanced Formal Reasoning", *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, pp. 41599–41633, 2025.

[50] L. de Moura and S. Ullrich, "The Lean 4 Theorem Prover and Programming Language", *Proc. International Conference on Automated Deduction (CADE)*, 2021, pp. 625–635.

[51] The mathlib Community, mathlib4: The Lean Mathematical Library (GitHub). https://github.com/leanprover-community/mathlib4.

[52] A. Ghose, J. Jang, A. B. Kahng and J. Lee, "AuDoPEDA source repository and archived evaluation snapshot," 2026. https://github.com/ABKGroup/AuDoPEDA; https://doi.org/10.5281/zenodo.22020200.

[53] V. A. Chhabria, V. Gopalakrishnan, A. B. Kahng, S. Kundu, Z. Wang, B.-Y. Wu and D. Yoon, "Strengthening the Foundations for IC Physical Design and ML EDA Research", *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024, pp. 1–9.

[54] K. Xu, D. Schwachhofer, J. Blocklove, I. Polian, P. Domanski, D. Pflüger, S. Garg et al., "Large Language Models (LLMs) for Electronic Design Automation (EDA)", *Proc. IEEE International System-on-Chip Conference (SOCC)*, 2025, pp. 1–6.

[55] The OpenROAD Project, "DPL Two-Pass with toggle," OpenROAD pull request #9038, 2026. https://github.com/The-OpenROAD-Project/OpenROAD/pull/9038.